МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Санкт-Петербургский политехнический
университет Петра Великого»**
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01_03 Технологии машинного обучения и искусственного интеллекта

Отчёт по лабораторной работе № 1 по дисциплине «Дискретная математика»

«Операции над мультимножествами и генерация бинарного кода Грея»

Выполнил:
Студент группы 5130201/50302 _____ Калинский В.И.
Принял:
Преподаватель _____ Востров А.В.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Математическое описание **5**

1.1 Понятие множества, мультимножества и универсума 5

1.2 Теоретико-множественные и арифметические операции 6

1.3 Алгоритм генерации бинарного кода Грея 7

2 Особенности реализации **9**

2.1 Архитектура программного комплекса и структуры данных . . 9

2.2 Спецификация функций и методов 9

3 Результаты работы **17**

3.1 Сценарий 1. Пустой универсум ($n = 0$) 17

3.2 Сценарий 2. Некорректный пользовательский ввод 18

3.3 Сценарий 3. Генерация универсума и заполнение мультимножеств 19

3.4 Сценарий 4. Теоретико-множественные операции (пункты меню 2–6) 22

3.5 Сценарий 5. Арифметические операции (пункты меню 7–10) 24

3.6 Сценарий 6. Большая разрядность кода грея и ограничение вывода 26

3.7 Сценарий 7. Заполнение мультимножеств по новой 26

Заключение **28**

Список литературы. **29**

Приложение А. Заголовочный файл MultiSet.hpp **30**

Приложение Б. Файл реализации MultiSet.cpp **32**

Приложение В. Файл точки входа и интерфейса main.cpp **35**

Приложение Г. Скрипт сборки CMakeLists.txt **44**

Введение

В современной дискретной математике и информатике классическая теория множеств часто оказывается недостаточной для описания реальных информационных структур. Во многих задачах возникает необходимость оперировать наборами элементов, в которых один и тот же объект может присутствовать многократно. Такие математические объекты называются *мультимножествами*.

Для практического применения мультимножеств в вычислительных системах требуется определить конечное базовое множество(носитель) и универсальное мультимножество(универсум), относительно которого корректно определяются операции на мультимножествах. В качестве эффективного базиса для формирования универсума в данной работе выступает двоичный код Грея.

Цель работы: изучение алгебраических и теоретико-множественных свойств мультимножеств, освоение алгоритмов генерации перестановок и кодов Грея, а также программная реализация консольного приложения для моделирования операций над мультимножествами.

Постановка задачи:

1. Разработать алгоритм и программный модуль генерации n-разрядного бинарного кода Грея, где разрядность $n \in [0, 27]$ задается пользователем. Верхняя граница разрядности выбрана исходя из критерия времени генерации до 5 секунд (при $n = 28$ среднее время составляет 7.3 с, при $n = 27$ – 3.6 с). Предусмотреть обработку краевого случая пустого универсума при $n = 0$.

2. Сформировать упорядоченный универсум U, кратности элементов которого генерируются случайным образом в диапазоне от 1 до 10 (при $n = 0$ универсум пуст).

3. Реализовать создание двух мультимножеств A и B над построенным носителем с мощностями, задаваемыми пользователем в диапазоне от 0 до доступной суммарной емкости универсума (не более 100).

4. Предусмотреть два альтернативных режима наполнения мультимножеств:

   - ручной ввод пользователем (выбор номера элемента носителя и задание его кратности с контролем непревышения как целевой мощности, так и доступной емкости элемента в универсуме);
   - автоматическая генерация кратностей элементов на основе датчика псевдослучайных чисел с контролем лимитов универсума.

5. Реализовать вычисление следующих операций над мультимножествами:

   - объединение ($A \cup B$);
   - пересечение ($A \cap B$);
   - теоретико-множественная разность ($A \setminus B$);
   - симметрическая разность ($A \triangle B$);
   - дополнение ($\overline{A} = U \setminus A$);
   - арифметическая сумма ($A + B$);
   - арифметическая разность ($A - B$ и $B - A$);
   - арифметическое произведение ($A \cdot B$);
   - арифметическое деление ($A : B$ и $B : A$).

6. Предусмотреть ограничение вывода мультимножеств в консоль, строгую фильтрацию некорректного пользовательского ввода и интерактивное меню управления.

1 Математическое описание

1.1 Понятие множества, мультимножества и универсума

Классическим *множеством* называется совокупность определенных, различимых объектов (элементов), мыслимая как единое целое, причем каждый элемент либо принадлежит множеству, либо не принадлежит ему.

Мультимножество над базовым множеством $X = \{x_1, x_2, \ldots, x_N\}$ записывается в виде:

$$\hat{A} = \{x_1^{\alpha_1}, x_2^{\alpha_2}, \ldots, x_N^{\alpha_N}\}, \tag{1}$$

где каждому элементу $x_i \in X$ сопоставляется $\alpha_i \in \mathbb{N} \cup \{0\}$, называемое *кратностью* (степенью вхождения) элемента в мультимножество, а $x_i^{\alpha_i}$ обозначает вхождение элемента x_i ровно α_i раз. Если $\alpha_i = 0$, элемент фактически отсутствует в мультимножестве. Пустое мультимножество обозначается вектором нулевых кратностей: $\varnothing = (0, 0, \ldots, 0)$.

Мощностью мультимножества $|\hat{A}|$ называется сумма кратностей всех его элементов:

$$|\hat{A}| = \sum_{i=1}^{N} \alpha_i. \tag{2}$$

Универсумом мультимножеств U называется фиксированное мультимножество над тем же носителем X:

$$U = \{x_1^{m_1}, x_2^{m_2}, \ldots, x_N^{m_N}\}, \quad m_i \geq 0, \tag{3}$$

ограничивающее сверху максимальную кратность элементов любого рассматриваемого мультимножества: $\forall i \ (\alpha_i \leq m_i)$.

В рамках разработанной программы универсум U формируется следующим образом: кратности его элементов m_i задаются псевдослучайными числами в диапазоне от 1 до 10:

$$U = \{x_1^{m_1}, x_2^{m_2}, \ldots, x_N^{m_N}\}, \quad m_i \in [1, 10]. \tag{4}$$

В краевом случае нулевой разрядности кода Грея ($n = 0$) носитель элементов отсутствует, универсум является пустым ($U = \varnothing$, $|U| = 0$), а мультимножества A и B автоматически инициализируются как пустые.

1.2 Теоретико-множественные и арифметические операции

Пусть $A = \{x_i^{\alpha_1}\}$ и $B = \{x_i^{\alpha_2}\}$ – два мультимножества над носителем X, а $U = \{x_i^{m_i}\}$ – универсум с кратностями $m_i \in [1, 10]$. Правила вычисления кратностей результирующих мультимножеств определяются следующим образом:

1. **Пересечение** $(A \cap B)$**:** Кратность каждого элемента равна минимальной из кратностей в операндах:

$$k_{A \cap B}(x_i) = \min(\alpha_{1,i}, \alpha_{2,i}). \tag{5}$$

2. **Объединение** $(A \cup B)$**:** Кратность каждого элемента равна максимальной из кратностей в операндах:

$$k_{A \cup B}(x_i) = \max(\alpha_{1,i}, \alpha_{2,i}). \tag{6}$$

3. **Дополнение** $(\overline{A})$**:** Дополнение мультимножества вычисляется относительно универсума U:

$$k_{\overline{A}}(x_i) = m_i - \alpha_{1,i}. \tag{7}$$

4. **Разность** $(A \setminus B)$**:** В теоретическом конспекте разность определяется через пересечение с дополнением: $A \setminus B = A \cap \overline{B}$. Подставляя формулу дополнения, получаем:

$$k_{A \setminus B}(x_i) = \min(\alpha_{1,i}, m_i - \alpha_{2,i}). \tag{8}$$

5. **Симметрическая разность** $(A \triangle B)$**:** Определяется как разность между объединением и пересечением мультимножеств:

$$A \triangle B = (A \cup B) \setminus (A \cap B). \tag{9}$$

Кратность элемента:

$$k_{A \triangle B}(x_i) = \min\Big(\max(\alpha_{1,i}, \alpha_{2,i}),\ m_i - \min(\alpha_{1,i}, \alpha_{2,i})\Big). \tag{10}$$

6. **Арифметическая сумма ($A + B$):** Кратность каждого элемента в сумме мультимножеств ограничивается сверху его максимальной кратностью в универсуме U:

$$k_{A+B}(x_i) = \min(\alpha_{1,i} + \alpha_{2,i},\ m_i). \tag{11}$$

7. **Арифметическая разность ($A - B$):** Кратность представляет собой усеченную разность (не может быть отрицательной):

$$k_{A-B}(x_i) = \max(0, \alpha_{1,i} - \alpha_{2,i}), \tag{12}$$

8. **Арифметическое произведение ($A \cdot B$):** Поэлементное произведение кратностей с обязательным ограничением сверху кратностью универсума U:

$$k_{A \cdot B}(x_i) = \min(\alpha_{1,i} \cdot \alpha_{2,i}, m_i). \tag{13}$$

9. **Арифметическое деление ($A : B$):** Целочисленное деление кратностей. В соответствии с договоренностью при делении на 0 результирующая кратность принимается равной 0:

$$k_{A/B}(x_i) = \begin{cases} [\alpha_{1,i}/\alpha_{2,i}], & \alpha_{2,i} \neq 0, \\ 0, & \alpha_{2,i} = 0. \end{cases} \tag{14}$$

1.3 Алгоритм генерации бинарного кода Грея

Бинарным кодом Грея порядка n называется последовательность всех 2^n двоичных векторов длины n, упорядоченная таким образом, что любые два соседних кода, а также первый и последний, отличаются ровно на один разряд.

Для генерации последовательности кодовых слов используется алгоритм, описанный в фундаментальном учебнике Ф. А. Новикова «Дискретная математика для программистов» [1]. Согласно данному алгоритму, переход от кодового слова B_{i-1} к следующему кодовому слову B_i ($i = 1, 2, \ldots, 2^n - 1$) осуществляется инверсией бита с индексом $p = Q(i) - 1$, где функция $Q(i)$ возвращает номер младшей единицы в двоичной записи шага i:

$$Q(i) = 1 + \max\{k \in \mathbb{N} \cup \{0\} \mid i \text{ делится на } 2^k\}. \tag{15}$$

Алгоритмически вычисление $Q(i)$ представляет собой подсчет числа делений i на 2 до получения нечетного остатка:

$$\begin{cases} q = 1, \quad j = i; \\ \text{пока } (j \pmod 2 == 0): \quad j = j/2, \quad q = q + 1; \\ \text{возврат } q. \end{cases} \tag{16}$$

Тогда закон формирования кодовой последовательности имеет вид:

$$B_0 = 0, \quad B_i = B_{i-1} \oplus (1 \ll (Q(i) - 1)), \quad i = 1, \ldots, 2^n - 1, \tag{17}$$

где $\oplus$ – побитовая операция XOR, $\ll$ – побитовый сдвиг влево. Вектор сгенерированных целых чисел служит базисом упорядоченного универсума.

Разрядность генерируемого кода ограничена диапазоном $n \in [0, 27]$. Выбор верхней границы обусловлен временным критерием: время построения носителя не должно превышать 5 секунд. В ходе экспериментальных замеров при $n = 28$ среднее время работы генератора составило 7.3 секунды, тогда как при $n = 27$ оно составило 3.6 секунды, что удовлетворяет установленному критерию быстродействия. При $n = 0$ генерация не выполняется, и универсум остаётся пустым.

2 Особенности реализации

2.1 Архитектура программного комплекса и структуры данных

Программный комплекс спроектирован на языке C++ (стандарт C++20) в соответствии с принципами модульности и разделения ответственности:

- `MultiSet.hpp` – спецификация интерфейса класса мультимножества и внешних операторов;
- `MultiSet.cpp` – реализация операций над мультимножествами с применением алгоритмов стандартной библиотеки шаблонов STL (`std::transform`);
- `main.cpp` – модуль консольного взаимодействия, генерации кода Грея, ввода данных и управления логикой;
- `CMakeLists.txt` – сценарий автоматизированной сборки проекта со статической линковкой библиотеки `mylib` и интеграцией динамического санитайзера памяти AddressSanitizer (`-fsanitize=address -fno-omit-frame-pointer -O1`).

Основной структурой данных для представления мультимножества выступает класс `MultiSet`, являющийся адаптером *std::vector<int>*. Внутреннее состояние объекта описывается вектором кратностей `std::vector<int> counts`.

Длина вектора равна текущей мощности носителя универсума $N = 2^{\text{bitrate}}$ (переменная `U_SIZE`). Индекс ячейки вектора i однозначно сопоставлен i-му кодовому слову кода Грея. Глобальный универсум хранится в переменной `MultiSet multi_universe`.

2.2 Спецификация функций и методов

В данном подразделе приведено описание всех функций программы с указанием их входов, выходов и алгоритмического принципа работы, сгруппированных по функциональному назначению.

Методы класса MultiSet

1. Конструктор `MultiSet::MultiSet()`

   - **Вход:** инициализация параметров функции.
   - **Выход:** инициализированный объект класса `MultiSet`.
   - **Описание:** конструктор по умолчанию; выделяет непрерывный блок памяти под вектор кратностей `counts` размера `U_SIZE` и заполняет его нулями.

2. `std::vector<int>::iterator MultiSet::begin()`

   - **Вход:** инициализация параметров функции.
   - **Выход:** итератор на начало вектора кратностей `counts`.
   - **Описание:** возвращает начальный итератор вектора кратностей.

3. `std::vector<int>::iterator MultiSet::end()`

   - **Вход:** инициализация параметров функции.
   - **Выход:** итератор на позицию за последним элементом вектора кратностей.
   - **Описание:** возвращает конечный итератор вектора кратностей.

4. `int MultiSet::size()`

   - **Вход:** инициализация параметров функции.
   - **Выход:** целое число типа `int` – размер вектора кратностей.
   - **Описание:** возвращает физическое количество ячеек во внутреннем векторе `counts`, равное мощности носителя универсума.

5. `void MultiSet::clear()`

   - **Вход:** инициализация параметров функции.
   - **Выход:** отсутствует (`void`).
   - **Описание:** полностью очищает внутренний вектор кратностей `counts`, обнуляя его размер.

6. `int& MultiSet::operator[](size_t pos)`

- **Вход:** индекс позиции `pos` типа `size_t`.
- **Выход:** ссылка `int&` на значение кратности элемента.
- **Описание:** предоставляет прямой доступ по индексу к кратности элемента мультимножества для чтения и модификации.

7. `void MultiSet::resize(size_t new_size, const int val)`

- **Вход:** новый размер `new_size` типа `size_t`, значение инициализации `val` типа `const int`.
- **Выход:** изменение внутреннего вектора кратностей.
- **Описание:** перераспределяет память контейнера под размер `new_size`, заполняя новые ячейки значением `val`.

8. `bool MultiSet::isBigSize()`

- **Вход:** инициализация параметров функции.
- **Выход:** превышает ли количество ненулевых кратностей 32 (тип bool).
- **Описание:** выполняет подсчет элементов с ненулевой кратностью ($\alpha_i > 0$); если их количество превышает либо равно 32, возвращает `true`, инициируя безопасное усечение консольного вывода.

Функции генерации, кодирования и валидации

9. `int Q(int i)`

- **Вход:** натуральное число $i \geq 1$ (номер шага генерации).
- **Выход:** целое число $q \geq 1$ – номер позиции младшей единицы.
- **Описание:** определяет номер позиции младшей единицы (начиная с 1) в двоичном представлении числа i путём циклического деления на 2 до получения нечетного остатка.

10. `void generateGrayCode(int n, vector<int> &target)`

- **Вход:** разрядность кода $n \in [0, 27]$, ссылка на вектор `target`.
- **Выход:** заполненный вектор кодовых слов `target` длины 2^n.
- **Описание:** формирует n-разрядную последовательность бинарного кода Грея, начиная с 0 и инвертируя бит с индексом $Q(i) - 1$ на каждом i-м шаге.

11. `int getValidatedInt(const string &prompt, int min_val, int max_val)`

    - **Вход:** строка приглашения `prompt`, нижняя граница `min_val`, верхняя граница `max_val`.
    - **Выход:** целое число типа `int` от пользователя.
    - **Описание:** выводит приглашение, считывает число и проверяет, попадает ли значение в диапазон $[\text{min_val}, \text{max_val}]$; в случае некорректного ввода очищает буфер и запрашивает число повторно.

12. `long long createUniverse(MultiSet &u)`

    - **Вход:** ссылка на объект универсума `u`.
    - **Выход:** суммарная ёмкость универсума типа `long long`.
    - **Описание:** изменяет размер контейнера `u` под `U_SIZE` $= 2^N$ и заполняет кратность каждого элемента случайным числом от 1 до 10.

Функции консольного отображения и наполнения множеств

13. `void printBinary(int code, int bitrate)`

    - **Вход:** числовой эквивалент кода `code`, разрядность `bitrate`.
    - **Выход:** прямой вывод в выходной поток .
    - **Описание:** побитово выводит целое число в двоичном формате заданной разрядности.

14. `void printMultiSet(MultiSet &ms, const vector<int> &u, int bitrate)`

    - **Вход:** ссылка на мультимножество `ms`, вектор кодовых слов универсума `u`, разрядность `bitrate`.
    - **Выход:** прямой вывод в поток `std::cout`.
    - **Описание:** выводит элементы с ненулевой кратностью в виде пар вида `'код': кратность`; при срабатывании метода `ms.isBigSize()` выводит первые 32 ненулевых элемента и символ многоточия.

15. `void fillManually(MultiSet &ms, const vector<int> &universe, int bitrate, int target_capacity)`

    - **Вход:** ссылка на мультимножество `ms`, вектор универсума, разрядность, целевая мощность множества.
    - **Выход:** модифицированное мультимножество `ms`.
    - **Описание:** поэлементный выбор элементов по номеру и ввод кратностей с контролем остатка емкости каждого элемента в универсуме U.

16. `void fillAutomatically(MultiSet &ms, const vector<int> &universe, int target_capacity)`

    - **Вход:** ссылка на мультимножество `ms`, вектор универсума, целевая мощность множества.
    - **Выход:** модифицированное мультимножество `ms`.
    - **Описание:** генерирует наполнение мультимножества, добавляя случайное число к $\Delta \in [1, \min(m_i - \alpha_i, \text{rem})]$ к случайному элементу мультимножества `ms` до достижения целевой мощности.

17. `int createMultiSets(MultiSet &A, MultiSet &B, vector<int> &universe)`

    - **Вход:** ссылки на мультимножества A, B и вектор носителя `universe`.
    - **Выход:** выбранная разрядность кода Грея `bitrate` типа `int`.
    - **Описание:** выполняет полный цикл начальной инициализации: очищает контейнеры, сбрасывает размер `U_SIZE`, запрашивает разрядность n, генерирует код Грея и универсум U, после чего запускает процедуры заполнения множеств A и B.

Операции над мультимножествами

18. `MultiSet uni(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – объединение $A \cup B$.

- **Описание:** вычисляет объединение по правилу $\max(a_i, b_i)$ с использованием алгоритма `std::transform`.

19. `MultiSet insertion(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – пересечение $A \cap B$.
    - **Описание:** вычисляет пересечение по правилу $\min(a_i, b_i)$ с использованием алгоритма `std::transform`.

20. `MultiSet complement(MultiSet &A)`

    - **Вход:** ссылка на мультимножество A.
    - **Выход:** объект `MultiSet` – дополнение $\overline{A} = U \setminus A$.
    - **Описание:** вычисляет разность кратностей универсума и множества A $(m_i - a_i)$ с помощью `std::transform`.

21. `MultiSet difference(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – теоретико-множественная разность $A \setminus B$.
    - **Описание:** реализует формулу $A \cap \overline{B}$ через композицию вызовов `insertion(A, complement(B))`.

22. `MultiSet sym_difference(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – симметрическая разность $A \triangle B$.
    - **Описание:** вычисляет разность объединения и пересечения: `difference(uni(A, B), insertion(A, B))`.

23. `MultiSet operator+(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – арифметическая сумма $A + B$.
    - **Описание:** в два прохода `std::transform` рассчитывает сумму кратностей $a_i + b_i$ и ограничивает результат емкостью универсума $\min(a_i + b_i, m_i)$.

24. `MultiSet operator-(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – арифметическая разность $A - B$.
    - **Описание:** вычисляет неотрицательную разность кратностей $\max(0, a_i - b_i)$ через `std::transform`.

25. `MultiSet operator*(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – арифметическое произведение $A \cdot B$.
    - **Описание:** поэлементно перемножает кратности с отсечением по емкости универсума $\min(a_i \cdot b_i, m_i)$.

26. `MultiSet operator/(MultiSet &A, MultiSet &B)`

    - **Вход:** ссылки на мультимножества A и B.
    - **Выход:** объект `MultiSet` – результат арифметического деления $A : B$.
    - **Описание:** выполняет целочисленное деление $\lfloor a_i / b_i \rfloor$; при нулевом делителе ($b_i = 0$) возвращает 0.

27. `int main(int argc, char const *argv[])`

    - **Вход:** аргументы командной строки (`argc`, `argv`).
    - **Выход:** код возврата процесса `0`.
    - **Описание:** точка входа в программу; выполняет запуск генератора псевдослучайных чисел, координирует вызовы подсистемы перезапуска `createMultiSets` и цикл интерактивного меню операций.

Полный исходный код каждого файла проекта вынесен в отдельные приложения:

- заголовочный файл `MultiSet.hpp` – в Приложении А (см. листинг 1);
- файл реализации операций `MultiSet.cpp` – в Приложении Б (см. листинг 2);

- файл главного модуля и TUI `main.cpp` – в Приложении В (см. листинг 3);
- скрипт системы сборки `CMakeLists.txt` – в Приложении Г (см. листинг 4).

3 Результаты работы

Для проверки разработанного программного комплекса проведены серии тестов, охватывающие как штатную обработку теоретико-множественных и арифметических операций, так и проверку отказоустойчивости при некорректном вводе данных и обработку граничных состояний (краевой случай нулевой разрядности, предельная разрядность носителя).

3.1 Сценарий 1. Пустой универсум ($n = 0$)

Пользователь задает нулевую разрядность кода Грея ($n = 0$). В этом случае базовый носитель элементов отсутствует ($N = 2^0 = 0$ кодов в логике программы), поэтому универсум является пустым. Программа выводит диагностическое сообщение: «`Универсум пуст (мощность равна 0). Множества A и B автоматически оставлены пустыми.`» и сразу переходит к главному меню без запроса мощностей и режимов заполнения.

При выборе пункта меню 1 на экран выводится статус отсутствия элементов во всех трех множествах. Выполнение любой из операций (пункты меню 2–10) не приводит к аварийному завершению программы или ошибкам сегментации (SEGV): циклы преобразования по нулевому диапазону корректно завершаются, и в качестве результата выводится пустое мультимножество $\varnothing$.

На рис. 1 представлен вывод консоли при вводе разрядности 0: зафиксированы сообщение об автоматической инициализации пустых множеств A, B и универсума, а также корректное отображение пустого состояния объединения множеств на примере вызова второго пункта меню.

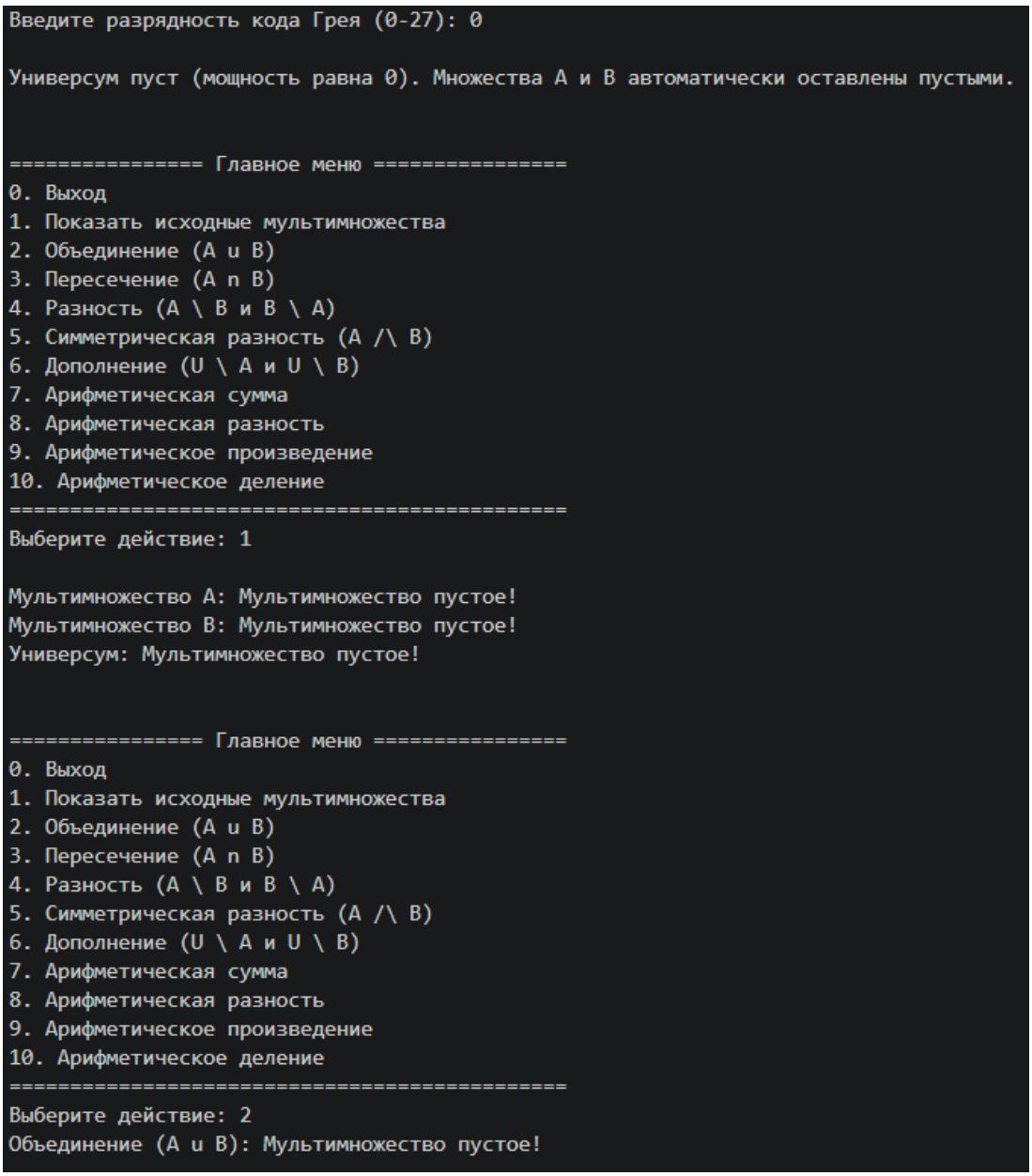


Рис. 1: Работа программы в режиме нулевой разрядности кода Грея ($n = 0$)

3.2 Сценарий 2. Некорректный пользовательский ввод

В рамках сценария проверяется реакция подсистемы валидации `getValidatedInt()`:

1. Ввод нечисловых символов (`"abc"`, `"!?"`) на этапе выбора разрядности;
2. Ввод отрицательного числа (-5 при допустимом минимуме 0);
3. Ввод числа, превышающего допустимый диапазон быстродействия (28 или 35 при максимуме 27);

4. Попытка задать кратность элемента при ручном заполнении, превышающую оставшуюся емкость данного элемента в сгенерированном универсуме U.

При нарушении формата потока ввода `cin` программа очищает флаги ошибок вызовом `cin.clear()`, освобождает входной буфер методом `cin.ignore()` и выводит сообщение с указанием корректных границ допустимого диапазона, предотвращая зацикливание.

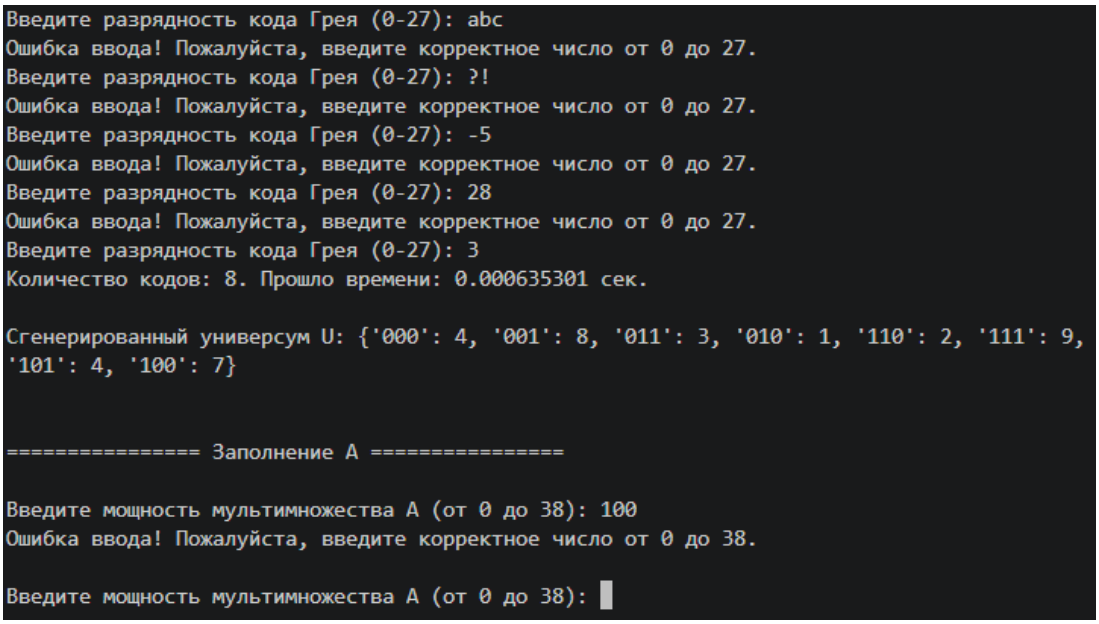


Рис. 2: Отработка защиты от некорректного пользовательского ввода

На рис. 2 показана реакция терминала: ввод символьных строк и чисел вне диапазона $[0, 27]$ блокируется с выводом диагностических предупреждений и повторным запросом ввода до получения корректного значения.

3.3 Сценарий 3. Генерация универсума и заполнение мультимножеств

Пользователь задает разрядность $n = 3$. Программа генерирует $N = 2^3 = 8$ комбинаций бинарного кода Грея. Пользователь задает мощности: $|A| = 5$ (ручное заполнение), $|B| = 8$ (автоматическое заполнение). При ручном заполнении множества A интерфейс отображает доступную кратность элементов в U и динамически ограничивает максимальный ввод. Множество

B наполняется автоматически функцией `fillAutomatically()` со случайным распределением кратностей с обязательным контролем наличия элементов в универсуме.

На рис. 3 показан вывод консоли при генерации 8 кодов Грея, случайном задании кратностей универсума, а также пошаговый процесс ручного ввода множества A с контролем лимитов емкости U и автоматической генерации множества B. Полученные мультимножества A и B показаны при вызове пункта 1 меню.

```
================ Заполнение A ================

Введите мощность мультимножества A (от 0 до 46): 5

Выберите способ заполнения множества A (1 - Вручную, 2 - Автоматически): 1

Доступные элементы универсума (код и максимальная кратность в U):
1. [000] — кратность в U: 3
2. [001] — кратность в U: 9
3. [011] — кратность в U: 6
4. [010] — кратность в U: 5
5. [110] — кратность в U: 4
6. [111] — кратность в U: 4
7. [101] — кратность в U: 7
8. [100] — кратность в U: 8

Текущая мощность: 0 из 5
Выберите номер элемента для добавления (от 1 до 8): 1
Введите кратность от 1 до 3 (оставшаяся ёмкость в U: 3): 1

Текущая мощность: 1 из 5
Выберите номер элемента для добавления (от 1 до 8): 3
Введите кратность от 1 до 4 (оставшаяся ёмкость в U: 6): 2

Текущая мощность: 3 из 5
Выберите номер элемента для добавления (от 1 до 8): 6
Введите кратность от 1 до 2 (оставшаяся ёмкость в U: 4): 1

Текущая мощность: 4 из 5
Выберите номер элемента для добавления (от 1 до 8): 8
Введите кратность от 1 до 1 (оставшаяся ёмкость в U: 8): 1
==============================================


================ Заполнение B ================

Введите мощность мультимножества B (от 0 до 46): 8

Выберите способ заполнения множества B (1 - Вручную, 2 - Автоматически): 2

Автоматическое заполнение выполнено!
==============================================


================ Главное меню ================
0. Выход
1. Показать исходные мультимножества
2. Объединение (A ∪ B)
3. Пересечение (A ∩ B)
4. Разность (A \ B и B \ A)
5. Симметрическая разность (A /\ B)
6. Дополнение (U \ A и U \ B)
7. Арифметическая сумма
8. Арифметическая разность
9. Арифметическое произведение
10. Арифметическое деление
==============================================
Выберите действие: 1

Мультимножество A: {'000': 1, '011': 2, '111': 1, '100': 1}
Мультимножество B: {'011': 4, '010': 1, '111': 3}
Универсум: {'000': 3, '001': 9, '011': 6, '010': 5, '110': 4, '111': 4, '101': 7, '100': 8}
```

Рис. 3: Протокол генерации универсума и наполнения мультимножеств A и B

3.4 Сценарий 4. Теоретико-множественные операции (пункты меню 2–6)

На полученных мультимножествах из предыдущих сценариев протестированы операции из пунктов меню 2–6. Полученные результаты строго согласуются с ручным теоретическим расчетом по формулам, приведенным в подразделе 1.2. На рис. 4 зафиксированы экранные снимки консоли при вызове пунктов меню 2–4, а на рис. 5 – пунктов 5 и 6 меню: для каждого пункта отображается правильный результат.

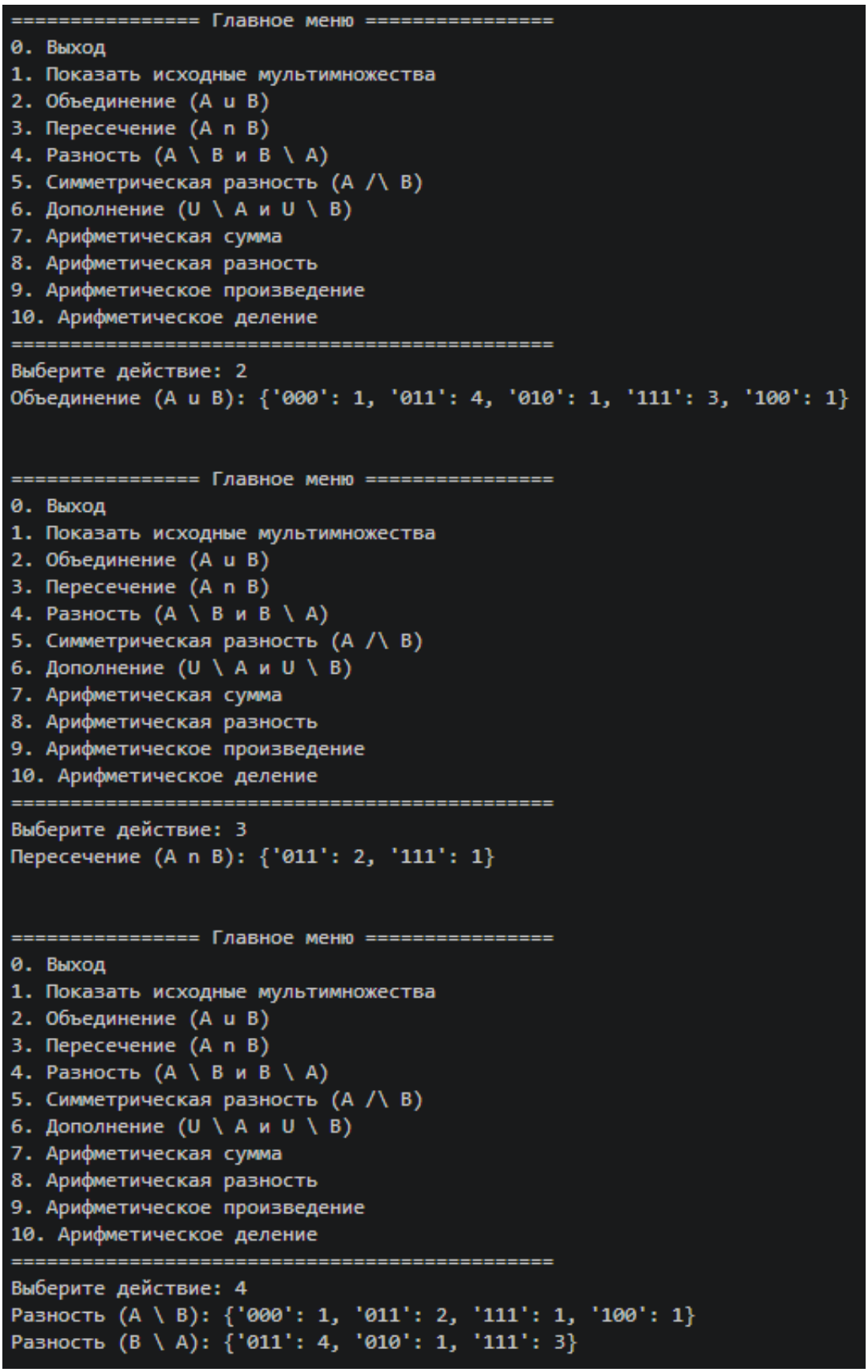


Рис. 4: Результаты выполнения теоретико-множественных операций (пункты меню 2–4)

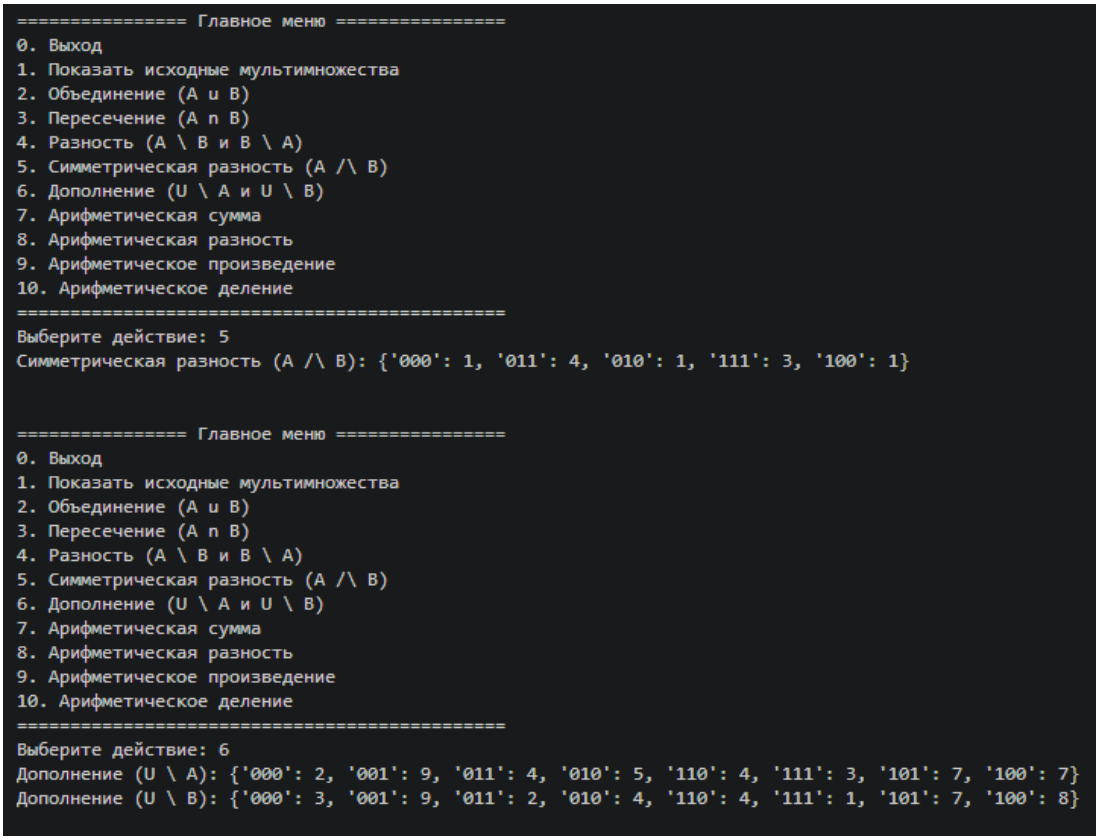


Рис. 5: Результаты выполнения теоретико-множественных операций (пункты меню 5 и 6)

3.5 Сценарий 5. Арифметические операции (пункты меню 7–10)

На полученных мультимножествах из предыдущих сценариев протестированы операции из пунктов меню 7–10. Полученные результаты строго согласуются с ручным теоретическим расчетом по формулам, приведенным в подразделе 1.2. На рис. 6 зафиксированы экранные снимки консоли при вызове пунктов меню 7–10: для каждого пункта отображается правильный результат.

```
Выберите действие: 7
Арифметическая сумма (A + B): {'000': 1, '011': 6, '010': 1, '111': 4, '100': 1}


================ Главное меню ================
0. Выход
1. Показать исходные мультимножества
2. Объединение (A ∪ B)
3. Пересечение (A ∩ B)
4. Разность (A \ B и B \ A)
5. Симметрическая разность (A /\ B)
6. Дополнение (U \ A и U \ B)
7. Арифметическая сумма
8. Арифметическая разность
9. Арифметическое произведение
10. Арифметическое деление
==============================================
Выберите действие: 8
Арифметическая разность (A - B): {'000': 1, '100': 1}
Арифметическая разность (B - A): {'011': 2, '010': 1, '111': 2}


================ Главное меню ================
0. Выход
1. Показать исходные мультимножества
2. Объединение (A ∪ B)
3. Пересечение (A ∩ B)
4. Разность (A \ B и B \ A)
5. Симметрическая разность (A /\ B)
6. Дополнение (U \ A и U \ B)
7. Арифметическая сумма
8. Арифметическая разность
9. Арифметическое произведение
10. Арифметическое деление
==============================================
Выберите действие: 9
Арифметическое произведение (A * B): {'011': 6, '111': 3}


================ Главное меню ================
0. Выход
1. Показать исходные мультимножества
2. Объединение (A ∪ B)
3. Пересечение (A ∩ B)
4. Разность (A \ B и B \ A)
5. Симметрическая разность (A /\ B)
6. Дополнение (U \ A и U \ B)
7. Арифметическая сумма
8. Арифметическая разность
9. Арифметическое произведение
10. Арифметическое деление
==============================================
Выберите действие: 10
Арифметическое деление (A / B): {}
Арифметическое деление (B / A): {'011': 2, '111': 3}
```

Рис. 6: Результаты выполнения арифметических операций (пункты меню 7–10)

3.6 Сценарий 6. Большая разрядность кода грея и ограничение вывода

Для проверки масштабируемости программы проведен запуск с повышенной разрядностью. При предельной разрядности $n = 27$ размер носителя составляет $N = 2^{27} = 134\,217\,728$ кодовых комбинаций. Зафиксированное время генерации составило 4.65122 секунды, что удовлетворяет требованию быстродействия до 5 секунд.

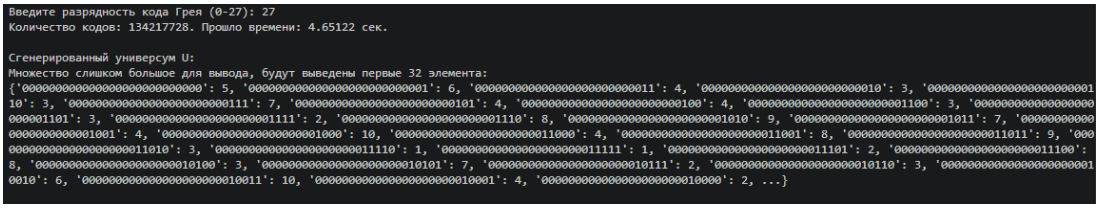


Рис. 7: Ограничение вывода до 32 элементов и замер времени генерации при больших n

На рис. 7 продемонстрирована работа механизма усечения вывода для размерностей более 32 элементов, а также замер времени работы генератора кода Грея при $n = 27$.

3.7 Сценарий 7. Заполнение мультимножеств по новой

Для удобства пользователя была добавлена функция заполнения универсума и мультимножеств заново без перезапуска программы. На рис. 8 продемонстрирована работа механизма перезапуска и нового заполнения мультимножеств.

```
Выберите действие: 0


========== Вы хотите начать заново? ==========
0. Выход
1. Начать заново
==============================================
Выберите действие: 1
Введите разрядность кода Грея (0-27): 2
Количество кодов: 4. Прошло времени: 1.0868e-05 сек.

Сгенерированный универсум U: {'00': 7, '01': 3, '11': 3, '10': 4}


=============== Заполнение A ===============

Введите мощность мультимножества A (от 0 до 17): 10

Выберите способ заполнения множества A (1 - Вручную, 2 - Автоматически): 2

Автоматическое заполнение выполнено!
==============================================


=============== Заполнение B ===============

Введите мощность мультимножества B (от 0 до 17): 5

Выберите способ заполнения множества B (1 - Вручную, 2 - Автоматически): 2

Автоматическое заполнение выполнено!
==============================================


================ Главное меню ================
0. Выход
1. Показать исходные мультимножества
2. Объединение (A u B)
3. Пересечение (A n B)
4. Разность (A \ B и B \ A)
5. Симметрическая разность (A /\ B)
6. Дополнение (U \ A и U \ B)
7. Арифметическая сумма
8. Арифметическая разность
9. Арифметическое произведение
10. Арифметическое деление
==============================================
Выберите действие: 1

Мультимножество A: {'00': 1, '01': 2, '11': 3, '10': 4}
Мультимножество B: {'11': 2, '10': 3}
Универсум: {'00': 7, '01': 3, '11': 3, '10': 4}
```

Рис. 8: Новое заполнение мультимножеств

Заключение

В ходе лабораторной работы на языке C++20 разработан программный комплекс для генерации бинарного кода Грея и выполнения полного набора операций над мультимножествами. В рамках реализации спроектирован класс `MultiSet` на базе вектора кратностей с поддержкой ручного и автоматического режима ввода, независимая генерация кратностей элементов универсума случайными числами от 1 до 10, а также корректная обработка краевого случая пустого универсума ($n = 0$). По критерию допустимого времени генерации (≤ 5 с) максимальная разрядность кода Грея зафиксирована равной $n = 27$ со средним временем построения, равным 3.6 секунды. В рамках разработанного программного модуля реализованы объединение, пересечение, теоретико-множественная и симметрическая разность, дополнение, а также арифметические сумма, разность, произведение и деление.

Достоинства реализованного решения:

- Высокая скорость доступа к элементам и линейная сложность вычисления операций над мультимножеством.
- Лаконичность и безопасность кода благодаря алгоритмам STL (`std::transform`)

Недостатки решения:

- Экспоненциальный расход памяти при росте разрядности кода Грея ($N = 2^n$).
- Для сильно разреженных мультимножеств использование `vector` становится неэффективным из-за большого количества нулевых кратностей.

Масштабирование алгоритма: Для работы с большими универсумами целесообразно заменить плотный вектор ассоциативным контейнером, реализованным на хэш-таблице и не хранящим нулевых элементов. Также для ускорения поэлементной обработки возможно применение параллельных вычислений.

Список литературы

[1] Новиков Ф. А. Дискретная математика для программистов : Учебник для вузов / Ф. А. Новиков. – 3-е изд. – СПб. : Питер, 2009. – 384 с. (Дата обращения – 10.10.2026)

Приложение А. Заголовочный файл MultiSet.hpp

Листинг 1: Заголовочный файл MultiSet.hpp

```
#ifndef SET_HPP
#define SET_HPP

#include <vector>

class MultiSet
{
    std::vector<int> counts;

    public:
    MultiSet();
    std::vector<int>::iterator begin()
    {
        return counts.begin();
    }
    int size()
    {
        return counts.size();
    }
    void clear()
    {
        counts.clear();
    }
    std::vector<int>::iterator end()
    {
        return counts.end();
    }
    int &operator[](size_t pos)
    {
        return counts[pos];
    }
    void resize(size_t new_size, const int val)
    {
        counts.resize(new_size, val);
    }
    bool isBigSize()
    {
        int count = 0;
        for (size_t i = 0; i < size(); i++)
        {
            if (count >= 32)
            return true;
            if (counts[i] > 0)
```

```
                count++;
            }
            return false;
        }
    };

    extern int U_SIZE;
    extern MultiSet multi_universe;

    MultiSet uni(MultiSet &A, MultiSet &B);
    MultiSet insertion(MultiSet &A, MultiSet &B);
    MultiSet difference(MultiSet &A, MultiSet &B);
    MultiSet sym_difference(MultiSet &A, MultiSet &B);
    MultiSet complement(MultiSet &A);

    MultiSet operator+(MultiSet &A, MultiSet &B);
    MultiSet operator-(MultiSet &A, MultiSet &B);
    MultiSet operator*(MultiSet &A, MultiSet &B);
    MultiSet operator/(MultiSet &A, MultiSet &B);

    #endif
```

Приложение Б. Файл реализации MultiSet.cpp

Листинг 2: Файл реализации MultiSet.cpp

```cpp
#include "MultiSet.hpp"
#include <algorithm>
#include <cmath>
#include <iostream>

using namespace std;

MultiSet::MultiSet() : counts(U_SIZE, 0) {}

// объединение: max(a, b)
MultiSet uni(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return max(a, b); });
    return result;
}

// пересечение: min(a, b)
MultiSet insertion(MultiSet &A, MultiSet &B)
{
    MultiSet result;
    transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
    { return min(a, b); });
    return result;
}

// дополнение: m - a, m - кратностьэлта- вуниверсуме
MultiSet complement(MultiSet &A)
{
    MultiSet result;
    transform(A.begin(), A.end(), multi_universe.begin(), result.begin(), [](int a, int
        m)
    { return m - a; });
    return result;
}

// разность: A n -B
// A \ B = A n (not B)
MultiSet difference(MultiSet &A, MultiSet &B)
{
    MultiSet notB = complement(B);
    return insertion(A, notB);
```

```
43      }
44
45      // симметрическаяразность: (A u B) \ (A n B)
46      MultiSet sym_difference(MultiSet &A, MultiSet &B)
47      {
48          MultiSet AuB = uni(A, B);
49          MultiSet AnB = insertion(A, B);
50          return difference(AuB, AnB);
51      }
52
53      // арифметическаясумма
54      MultiSet operator+(MultiSet &A, MultiSet &B)
55      {
56          MultiSet result;
57          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
58          { return a + b; });
59
60          transform(result.begin(), result.end(), multi_universe.begin(), result.begin(), [](
                int sum, int m)
61          { return min(sum, m); });
62          return result;
63      }
64
65      // арифметическаяразность
66      MultiSet operator-(MultiSet &A, MultiSet &B)
67      {
68          MultiSet result;
69          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
70          { return max(0, a - b); });
71          return result;
72      }
73
74      // арифметическоепроизведение
75      MultiSet operator*(MultiSet &A, MultiSet &B)
76      {
77          MultiSet result;
78          for (int i = 0; i < U_SIZE; i++)
79          {
80              result[i] = min(A[i] * B[i], multi_universe[i]);
81          }
82          return result;
83      }
84
85      // арифметическоеделение
86      MultiSet operator/(MultiSet &A, MultiSet &B)
87      {
```

```
88          MultiSet result;
89          transform(A.begin(), A.end(), B.begin(), result.begin(), [](int a, int b)
90          { return (b == 0) ? 0 : (a / b); });
91          return result;
92      }
```

Приложение В. Файл точки входа и интерфейса main.cpp

Листинг 3: Файл точки входа и интерфейса main.cpp

```
1   //
        РеализоватьпрограммугенерациибинарногокодаГреядлязаполненияуниверсумамультмножествзаданной
        ( пользователемразрядности).
        Наосновеуниверсумаформируютсядвамультимножествадвумяспособамизаполнения-
        вручнуюиавтоматическивыбирает( пользователь). Мощностимножествзадаетпользователь.
        Врезультатенаэкранвыводятсярезультатыдействийнадмножествами: объединение,
        пересечение, разностьоба( варианта), симметрическаяразность, дополнениеоба(
        варианта), арифметическиесумма, разность, произведениеиделение. Крометого,
        требуетсяреализоватьзащитуотнекорректногопользовательскоговводаипользовательскоеменю
        .
2   // input_number()
3   // input_sequence()
4
5   // union()
6   // insertion()
7   // difference()
8   // sym_difference()
9   // complement()
10  // operator+()
11
12  #include <string>
13  #include <limits>
14  #include <iostream>
15  #include <vector>
16  #include <bitset>
17  #include <format>
18  #include <algorithm>
19  #include <chrono>
20  #include "MultiSet.hpp"
21
22  int U_SIZE = 0;
23  MultiSet multi_universe;
24
25  using namespace std;
26  int getValidatedInt(const string &prompt = "", int min_val = 0, int max_val =
        numeric_limits<int>::max())
27  {
28      int value;
29      while (true)
30      {
31          cout << prompt;
32          if (cin >> value && value >= min_val && value <= max_val)
33          {
```

```
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Ошибка ввода! Пожалуйста, введитекорректноечислоот" << min_val << " до"
            << max_val << ".\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

int Q(int i)
{
    int q = 1;
    int j = i;
    while (j % 2 == 0)
    {
        j = j / 2;
        q = q + 1;
    }
    return q;
}

void generateGrayCode(int n, vector<int> &target)
{
    target.clear();
    int B = 0;
    target.push_back(B);

    int total_steps = 1 << n;

    for (int i = 1; i < total_steps; i++)
    {
        int p_index = Q(i) - 1;
        B ^= 1 << p_index;
        target.push_back(B);
    }
}

void printBinary(int code, int bitrate)
{
    for (int i = bitrate - 1; i >= 0; i--)
    {
        cout << ((code >> i) & 1);
    }
}

```

```
79  void printMultiSet(MultiSet &ms, const vector<int> &u, int bitrate)
80  {
81      if (ms.size() == 0)
82      {
83          cout << "Мультимножество пустое!\n";
84          return;
85      }
86
87      size_t limit = u.size();
88      bool is_truncated = ms.isBigSize();
89
90      if (is_truncated)
91      {
92          cout << "\Множествоn слишкомбольшоедлявывода, будутвыведеныпервые32 элемента:\n";
93          limit = 32;
94      }
95
96      bool first = true;
97      cout << "{";
98          int count = 0;
99          for (size_t i = 0; count < limit && i < u.size(); ++i)
100         {
101             if (ms[i] > 0)
102             {
103                 if (!first)
104                 cout << ", ";
105                 cout << "'";
106                 printBinary(u[i], bitrate);
107                 cout << "': " << ms[i];
108                 first = false;
109                 count++;
110             }
111         }
112
113         if (is_truncated)
114         {
115             if (!first)
116             cout << ", ...";
117             else
118             cout << "...";
119         }
120         cout << "}\n";
121 }
122
123 long long createUniverse(MultiSet &u)
124 {
```

```
125         u.resize(U_SIZE, 0);
126         if (U_SIZE == 0)
127         return 0;
128         long long total_capacity = 0;
129         for (int i = 0; i < U_SIZE; ++i)
130         {
131             u[i] = 1 + rand() % 10;
132             total_capacity += u[i];
133         }
134         return total_capacity;
135     }
136
137     void fillManually(MultiSet &ms, const vector<int> &universe, int bitrate, int
            target_capacity)
138     {
139         int current_capacity = 0;
140
141         cout << "\Доступныеn элементыуниверсумакод( имаксимальнаякратностьвU):\n";
142         for (int i = 0; i < universe.size(); ++i)
143         {
144             cout << i + 1 << ". [";
145             printBinary(universe[i], bitrate);
146             cout << "] -кратностьвU: " << multi_universe[i] << "\n";
147         }
148
149         while (current_capacity < target_capacity)
150         {
151             cout << "\Текущаяn мощность: " << current_capacity << " из" << target_capacity <<
                    "\n";
152             int idx = getValidatedInt(format("Выберите номерэлементадлядобавленияот( {} до{})
                    : ", 1, universe.size()), 1, universe.size()) - 1;
153
154             int remaining_in_u = multi_universe[idx] - ms[idx];
155             if (remaining_in_u <= 0)
156             {
157                 cout << "Этот элементнедоступеноставшаяся( ёмкостьвуниверсумеравна0).
                        Выберитедругойэлемент.\n";
158                 continue;
159             }
160
161             int max_allowed = min(remaining_in_u, target_capacity - current_capacity);
162             string prompt = format("Введите кратностьот1 до{} оставшаяся( ёмкостьвU: {}): ",
                    max_allowed, remaining_in_u);
163             int count = getValidatedInt(prompt, 1, max_allowed);
164
165             ms[idx] += count;
```

```
166             current_capacity += count;
167         }
168 
169         cout << "=============================================\n";
170     }
171 
172     void fillAutomatically(MultiSet &ms, const vector<int> &universe, int target_capacity)
173     {
174         int current_capacity = 0;
175         while (current_capacity < target_capacity)
176         {
177             int idx = rand() % U_SIZE;
178             int max_delta = min(multi_universe[idx] - ms[idx], target_capacity -
                    current_capacity);
179             if (max_delta <= 0)
180             continue;
181             int delta = 1 + rand() % max_delta;
182             ms[idx] += delta;
183             current_capacity += delta;
184         }
185         cout << "\Автоматическоеn заполнениевыполнено! \n";
186         cout << "=============================================\n";
187     }
188 
189     int createMultiSets(MultiSet &A, MultiSet &B, vector<int> &universe)
190     {
191         // приbitrate = 28 времявычислениякодагреясоставиловсреднем7.3 секунды. при27 - 3.6
                сек.
192         A.clear();
193         B.clear();
194         universe.clear();
195         multi_universe.clear();
196         U_SIZE = 0;
197         int bitrate = getValidatedInt("Введите разрядностькодаГрея(0-27): ", 0, 27);
198         auto start = chrono::steady_clock::now();
199         if (bitrate == 0)
200         {
201             cout << "\Универсумn пустмощность( равна0). МножестваA иB
                    автоматическиоставленыпустыми.\n";
202         }
203         else
204         {
205             universe.resize(1 << bitrate, 0);
206             generateGrayCode(bitrate, universe);
207             U_SIZE = universe.size();
208 
```

```
209             A.resize(U_SIZE, 0);
210             B.resize(U_SIZE, 0);
211             long long total_u_capacity = createUniverse(multi_universe);
212 
213             auto end = chrono::steady_clock::now();
214             chrono::duration<double> elapsed = end - start;
215             cout << "Количество кодов: " << U_SIZE << ". Прошловремени: " << elapsed.count()
                    << " сек.\n";
216 
217             cout << "\Сгенерированныйn универсумU: ";
218             printMultiSet(multi_universe, universe, bitrate);
219 
220             int lower_capacity = 0;
221             int upper_capacity;
222             if (100LL < total_u_capacity)
223             upper_capacity = 100;
224             else
225             upper_capacity = total_u_capacity;
226 
227             cout << "\n\n================ ЗаполнениеA ================\n";
228             int capacityA = getValidatedInt(format("\Введитеn мощностьмультимножестваA от( {}
                     до{}): ", lower_capacity, upper_capacity),
229             lower_capacity, upper_capacity);
230 
231             if (capacityA > 0)
232             {
233                 int choiceA = getValidatedInt("\Выберитеn способзаполнениямножестваA (1 -
                        Вручную, 2 - Автоматически): ", 1, 2);
234                 if (choiceA == 1)
235                 fillManually(A, universe, bitrate, capacityA);
236                 else
237                 fillAutomatically(A, universe, capacityA);
238             }
239 
240             cout << "\n\n================ ЗаполнениеB ================\n";
241             int capacityB = getValidatedInt(format("\Введитеn мощностьмультимножестваB от( {}
                     до{}): ", lower_capacity, upper_capacity),
242             lower_capacity, upper_capacity);
243 
244             if (capacityB > 0)
245             {
246                 int choiceB = getValidatedInt("\Выберитеn способзаполнениямножестваB (1 -
                        Вручную, 2 - Автоматически): ", 1, 2);
247                 if (choiceB == 1)
248                 fillManually(B, universe, bitrate, capacityB);
249                 else
```

```
250             fillAutomatically(B, universe, capacityB);
251         }
252     }
253     return bitrate;
254 }
255 int main(int argc, char const *argv[])
256 {
257     srand(time(nullptr));
258     MultiSet A, B;
259     vector<int> universe;
260     int bitrate = createMultiSets(A, B, universe);
261     while (true)
262     {
263         cout << "\n\n=============== Главноеменю===============\n";
264         cout << "0. Выход\n";
265         cout << "1. Показатьисходныемультимножества\n";
266         cout << "2. Объединение(A u B)\n";
267         cout << "3. Пересечение(A n B)\n";
268         cout << "4. Разность(A \\ B иB \\ A)\n";
269         cout << "5. Симметрическаяразность(A /\\ B)\n";
270         cout << "6. Дополнение(U \\ A иU \\ B)\n";
271         cout << "7. Арифметическаясумма\n";
272         cout << "8. Арифметическаяразность\n";
273         cout << "9. Арифметическоепроизведение\n";
274         cout << "10. Арифметическоеделение\n";
275         cout << "==========================================\n";
276 
277         int menuChoice = getValidatedInt("Выберите действие: ", 0, 10);
278 
279         if (menuChoice == 0)
280         {
281             cout << "\n\n========== Выхотитеначатьзаново? ==========\n";
282             cout << "0. Выход\n";
283             cout << "1. Начатьзаново\n";
284             cout << "==========================================\n";
285             menuChoice = getValidatedInt("Выберите действие: ", 0, 1);
286             if (menuChoice == 1)
287             {
288                 bitrate = createMultiSets(A, B, universe);
289                 continue;
290             }
291             else
292             {
293                 cout << "Программа завершена.\n";
294                 break;
295             }
```

```
296             }
297
298             MultiSet result;
299
300             switch (menuChoice)
301             {
302                 case 1:
303                 cout << "\Мультимножествоn A: ";
304                 printMultiSet(A, universe, bitrate);
305                 cout << "Мультимножество B: ";
306                 printMultiSet(B, universe, bitrate);
307                 cout << "Универсум: ";
308                 printMultiSet(multi_universe, universe, bitrate);
309                 break;
310                 case 2:
311                 result = uni(A, B);
312                 cout << "Объединение (A u B): ";
313                 printMultiSet(result, universe, bitrate);
314                 break;
315                 case 3:
316                 result = insertion(A, B);
317                 cout << "Пересечение (A n B): ";
318                 printMultiSet(result, universe, bitrate);
319                 break;
320                 case 4:
321                 result = difference(A, B);
322                 cout << "Разность (A \\ B): ";
323                 printMultiSet(result, universe, bitrate);
324
325                 result = difference(B, A);
326                 cout << "Разность (B \\ A): ";
327                 printMultiSet(result, universe, bitrate);
328                 break;
329                 case 5:
330                 result = sym_difference(A, B);
331                 cout << "Симметрическая разность(A /\\ B): ";
332                 printMultiSet(result, universe, bitrate);
333                 break;
334                 case 6:
335                 result = complement(A);
336                 cout << "Дополнение (U \\ A): ";
337                 printMultiSet(result, universe, bitrate);
338
339                 result = complement(B);
340                 cout << "Дополнение (U \\ B): ";
341                 printMultiSet(result, universe, bitrate);
```

```
342             break;
343             case 7:
344             result = A + B;
345             cout << "Арифметическая сумма(A + B): ";
346             printMultiSet(result, universe, bitrate);
347             break;
348             case 8:
349             result = A - B;
350             cout << "Арифметическая разность(A - B): ";
351             printMultiSet(result, universe, bitrate);
352 
353             result = B - A;
354             cout << "Арифметическая разность(B - A): ";
355             printMultiSet(result, universe, bitrate);
356             break;
357             case 9:
358             result = A * B;
359             cout << "Арифметическое произведение(A * B): ";
360             printMultiSet(result, universe, bitrate);
361             break;
362             case 10:
363             result = A / B;
364             cout << "Арифметическое деление(A / B): ";
365             printMultiSet(result, universe, bitrate);
366 
367             result = B / A;
368             cout << "Арифметическое деление(B / A): ";
369             printMultiSet(result, universe, bitrate);
370             break;
371         }
372     }
373 
374     return 0;
375 }
```

Приложение Г. Скрипт сборки CMakeLists.txt

Листинг 4: Скрипт сборки CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
set (CMAKE_CXX_STANDARD 20)
project(lab1 LANGUAGES CXX)

set(CMAKE_CXX_FLAGS "-fsanitize=address -fno-omit-frame-pointer -O1")

add_library(mylib STATIC
src/MultiSet.hpp
src/MultiSet.cpp
)

add_executable(main
src/main.cpp
)

target_link_libraries(main mylib)

file(GENERATE
OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/run_test.sh"
CONTENT "cat ../test_data.txt | ./main\n"
)
```